Subject: How to Reproduce GPT-2 (124M) from Scratch (Part 3)

Date: from April 2, 2026 to April 13, 2026

Contents

1 Optimizer **1**

2 Batch Size Increase **3**

3 Weight Decay **3**

4 Gradient Accumulation **5**

5 Distributed Data Parallel **5**

6 Final Training Loss – Overfitting **7**

7 Training Dataset **8**

8 Training **8**

9 HellaSwag for Evaluation **9**

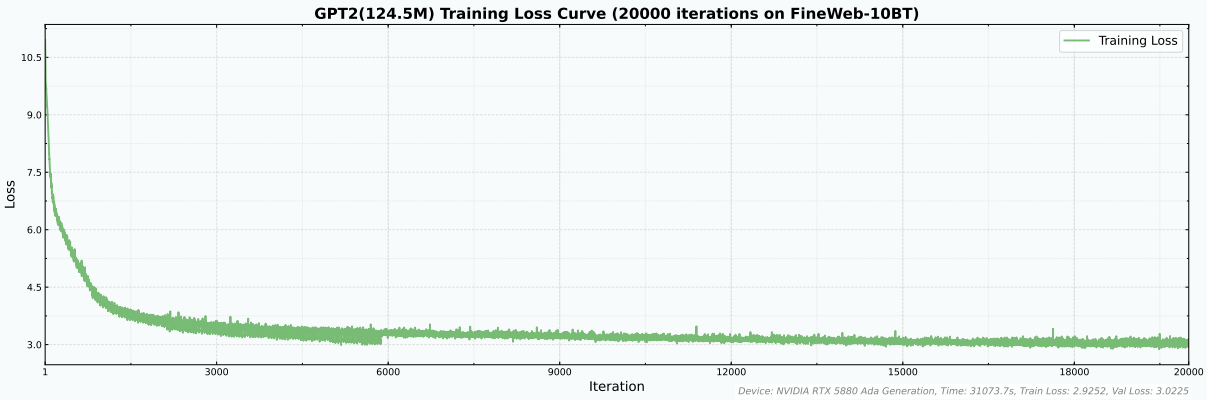
10 Final Results **9**

How to Reproduce GPT-2 (124M) (Part 2)

YouTube link：Let's reproduce GPT-2 (124M)

Introduction

在上一节笔记中，我们实现了通过 mixed precision 等技术加速模型训练，在本节笔记中我们将主要关注算法层面的优化，例如优化器中的超参数选择和 learning rate schedule，weight decay，gradient accumulation 等.
源代码可见 Andrej Karpathy 的 Nano-GPT 和 Nano-Chat.



Note

GPT-2[1] 开源了模型和相关的权重，但是原文和代码中并没有详细披露超参的选择；而 GPT-3[2] 虽未开源模型但原文中提供了详细的超参数配置. 由于二者模型架构的改变并不大，因此我们将进一步在 GPT-2 的代码中使用 GPT-3 的超参数.

1 Optimizer

Hyperparameter and Gradient Clipping

在默认的 `AdamW` 中，超参数为 $\beta_1 = 0.9, \beta_2 = 0.99$，这里我们参考 GPT-3 将超参数修改为：

```
learning_rate = 1e-4
betas = (0.9, 0.95)
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate, betas=betas,
    eps=1e-8)
```

同时，GPT-3 中使用了梯度裁剪 (Gradient Clipping)技术. 由于 Transformer 架构非常深，在反向传播时，梯度可能会因为连乘效应变得巨大，因此对梯度进行按比例放缩：

$$G_{new} = G \times \frac{\text{threshold}}{\max(\text{threshold}, \|G\|)}, \quad \text{threshold} = 1. \tag{1}$$

```
...
loss.backward() # Backpropagation
norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimier.step()
...
```

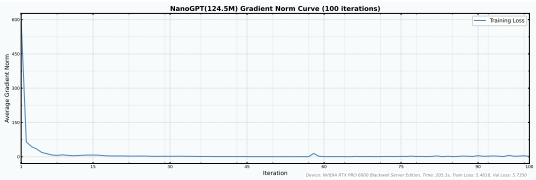


Figure 1: 无 clipping 的梯度情况，初期梯度很大 (超过 500).

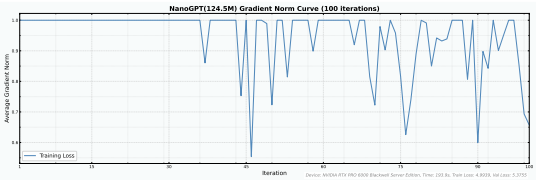


Figure 2: 有 clipping 的梯度情况，梯度稳定，均不超过 1.

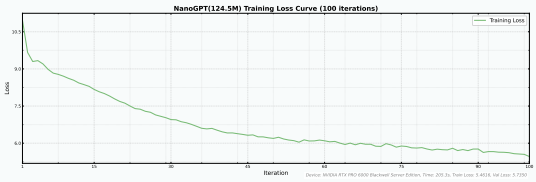


Figure 3: 无 clipping，final loss 5.7350.

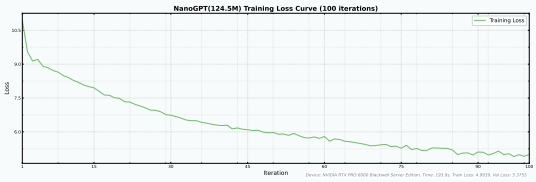


Figure 4: 有 clipping，final loss 5.3755.

Learning Rate Schedule

在训练过程中，为避免在训练后期因步长/学习率 (learning rate, lr) 过大导致 loss 来回震荡无法收敛的情况，一般会设置一个逐步减小 lr 的 lr schedule. 传统的等速率学习率 (Constant LR) 或阶梯式下降 (Step Decay) 容易让模型在后期陷入局部最优解，或者在最优解附近剧烈震荡，而 GPT-3 中使用的是 *cosine decay learning rate schedule with warmup*.

Cosine Decay LR Schedule with Warmup in GPT-3[2]

Step 1: 线性预热 (Linear Warmup). GPT-3 在前 375 million tokens 中进行了 linear LR warmup，将学习率逐渐线性提高至最大学习率.
Step 2: 余弦退火 (Cosine Decay). 在前 375 million tokens 之后，GPT-3 从最大值 ($\eta_{max} = 6 \times 10^{-4}$) 开始，按照余弦函数衰减:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})\left(1 + \cos\left(\frac{\pi t}{T}\right)\right), \quad t \triangleq \text{ epoch}, T \triangleq \text{ total epoch}. \tag{2}$$

Step 3: 最小学习率 (Minimum LR). 在达到最大学习率的 10% 后保持此(最小)学习率不变.

```python
max_iters = 100
lr_max = 3e-4
lr_min = lr_max * 0.1
warm_steps = 10

def get_lr(step):
    if step < warm_steps:
        return lr_max * (step+1) / warm_steps
    if step > max_iters - warm_steps:
        return lr_min
    decay_ratio = (step - warm_steps) / (max_iters - warm_steps)
    assert 0 <= decay_ratio <= 1, "Decay ratio should be between 0 and 1"
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + torch.cos(torch.tensor(
        decay_ratio * math.pi)))

    ...
    lr = get_lr(step)
    for param_group in optimizer.param_groups:
```

```
        param_group['lr'] = lr
    optimizer.step() # Update model parameters
    ...
```

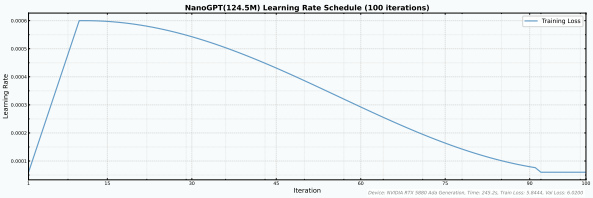


Figure 5: Learning Rate Schedule

Figure 6: Training Loss

2 Batch Size Increase

Gradual Batch Size Increase

GPT-3 中提到 *"We also gradually increase the batch size linearly from a small value (32k tokens) to the full value over the first 4-12 billion tokens of training, depending on the model size."* 但是本节中将不考虑这一部分. 简单来说逐渐增大 batch size 是因为模型初期梯度不稳定，因此使用较小的 batch size 节省算力让模型进入训练稳定阶段.

3 Weight Decay

Weight Decay

Weight decay 是一种常规的正则化方法，直观上来说其通过梯度更新中只继承上一步梯度的部分信息来防止梯度过度增长:

$$\theta_{next} = \theta - \eta \cdot \text{grad} \longrightarrow \theta_{next} = (1 - decay_rate)\theta - \eta \cdot \text{grad}. \tag{3}$$

对于所有的参数而言，实际上有一些维度为 1 的参数并不需要进行 weight decay，例如 Layer Norm 中的 γ (scale) 和 β (shift/bias)，因为一方面这些参数很小导致过拟合风险很小，另一方面衰减 γ, β 是在干预模型的标准化过程，这会破坏模型的数值稳定性.

因此我们重新对参数进行分类，对于需要进行 weight decay 的参数赋予超参数 `weight_decay` = `weight_decay`，对于不需要的参数赋予 `weight_decay` = 0.

```
class GPT2(nn.Module):
    ...
    def configure_optimizers(self, weight_decay, learning_rate, device):
        param_dict = {name: param for name, param in self.named_parameters()}
        param_dict = {name: param for name, param in param_dict.items() if param.
            requires_grad}
        decay_params = [p for name, p in param_dict.items() if p.dim() >= 2]
        no_decay_params = [p for name, p in param_dict.items() if p.dim() < 2]
        optim_groups = [
            {'params': decay_params, 'weight_decay': weight_decay},
            {'params': no_decay_params, 'weight_decay': 0.0}
        ]
        num_decay = sum(p.numel() for p in decay_params)
        num_no_decay = sum(p.numel() for p in no_decay_params)
        print(f"[INFO] decayed parameter tensors: {len(decay_params)} with {
            num_decay/1e6:.1f}M params")
```

```python
        print(f"[INFO] non-decayed parameter tensors: {len(no_decay_params)} with {num_no_decay/1e6:.1f}M params")
        fused_available = 'fused' in inspect.signature(torch.optim.AdamW).parameters # Check if fused AdamW is available in the current PyTorch version
        use_fused = fused_available and 'cuda' in device
        print(f"[INFO] Using fused AdamW optimizer: {use_fused}")
        optimizer = torch.optim.AdamW(optim_groups, lr=learning_rate, betas=[0.9, 0.95], eps=1e-8, fused=use_fused)
        return optimizer
    ...

optimizer = model.configure_optimizers(weight_decay=0.1, learning_rate=lr_max, device=device)
```

Outputs

[INFO] Decayed parameter tensors: 50 with 124.4M params
[INFO] Non-decayed parameter tensors: 98 with 0.1M params
[INFO] Using fused AdamW optimizer: True

Weight Decay

直观上看，weight decay 不断对历史梯度信息进行递减式继承，可以防止梯度过度增长. 事实上在 SGD 更新中其等价于 L_2 正则化：

$$\nabla(J+\frac{\lambda}{2}\theta^2)=\nabla J+\lambda\theta \Rightarrow \theta_{next}=\theta-\eta\cdot(\nabla J+\lambda\theta)=\underbrace{(1-\eta\lambda)\cdot\theta}_{\text{Weight Decay}}-\underbrace{\eta\cdot\nabla J}_{\text{Gradient Update}}. \tag{4}$$

但对于 Adam 二者并不等价. 由于 Adam 会根据梯度的一阶和二阶矩自动调整学习率：

$$\theta_{t+1}=\theta_t-\frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t,\quad m_t=\beta_1 m_{t-1}+(1-\beta_1)g_t, v_t=\beta_2 v_{t-1}+(1-\beta_2)g_t^2. \tag{5}$$

因此正则化项产生的梯度 $\lambda\theta$ 会被缩放为 $\frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\cdot(\lambda\theta_t)$，因此对于梯度较大 ($v_t$ 大) 的维度，正则化力度会变小，而梯度较小 (v_t 小) 的维度，正则化力度会变大. 因此目前的 Adam 优化器事实上都是 AdamW [3].

对于 weight decay 的一些其他理解：

1. 优胜劣汰: 如果一个参数对降低 loss 没有持续的贡献 (即 ∇J 长期趋近于 0)，那么在每一轮迭代中它都会被强行削减一点.
2. 控制系统的"总能量": 将权重平方和 $\sum\theta^2$ 理解为整个模型的总动能，训练过程中梯度更新会不断向系统"注入能量"，导致权重向量的模长 (norm) 无限增大，而 weight decay 通过不断地按比例抽走权重的数值，强制将模型的总能量控制在一个有限的半径内，从而确保模型是在一个"有界的球体"内寻找解.
3. 提高神经元"灵敏度"：Transformer 中包含大量 Sigmoid、Tanh 或 Softmax 操作，如果权重θ 变得非常大，激活函数的输入就会进入饱和区 (导数接近 0 的地方)，因此通过把权重往 0 推，它迫使参数停留在激活函数导数较大、最敏感的线性区域附近.
4. 消除"初始化"随机偏差: 随着训练进行，那些初始化带来的、对最终任务没有帮助的随机数值，会被 weight decay 逐渐"洗掉".

4 Gradient Accumulation

Gradient Accumulation

在实际的 GPT 预训练阶段中，batch size 都非常大 (例如 GPT-3 Small $\texttt{batch_size} = 0.5\text{M}$). 注意这里的 batch size 是指一次性处理的 token 数而不仅是传统 DL 中的 batch 数，例如目前我们的 batch size 是 $B \times T = 32 \times 1024 = 32768 \approx 0.03\text{M}$. 因此为了复刻 $\texttt{batch_size} = 0.5\text{M}$，一种方式是一次性处理接近 500 条 sample，但是这一般会超出个人使用的 GPU 负荷. 另一种方式是采用梯度累积 (Gradient Accumulation)，即通过累加几步 (小 $\texttt{batch_size}$ B) 迭代产生的梯度以模拟一步 (大 $\texttt{batch_size}$ B) 迭代，这样一次迭代步的 "batch size" 就为：

$$\texttt{batch_size} \times \texttt{block_size} \times \texttt{accumulated_steps} \tag{6}$$

```python
total_batch_size = 524288 # 2**19, ~0.5M tokens
assert total_batch_size % (batch_size * block_size) == 0, "Total batch size must be divisible by (batch_size * block_size)"
acc_steps = total_batch_size // (batch_size * block_size)
print(f"[INFO] Batch size: {batch_size} | Block size: {block_size}")
print(f"[INFO] Gradient Accumulation Steps: {acc_steps} | Total batch size: {total_batch_size} tokens")
```

Outputs

[INFO] Batch size: 32 — Block size: 1024
[INFO] Gradient Accumulation Steps: 16 — Total batch size: 524288 tokens

此处需要注意的一点是，在单个训练 step 中累积 gradient k 次后需要将其取平均. 若不取平均 (例如下面的代码) 会使得得出的 loss 累加方大，由于梯度计算的线性性也会使梯度累加放大：

```python
for micro_step in range(acc_steps):
    xb, yb = get_batch('train')
    logits, loss = model(xb, yb)
    loss.backward() # Backpropagation
```

正确的方式是取平均，计算我们真正模拟的单个 step 中一次性处理的 batch 的平均梯度：

```python
for micro_step in range(acc_steps):
    xb, yb = get_batch('train')
    with torch.autocast(device_type=device, dtype=torch.bfloat16): # Mixed precision for faster training
        logits, loss = model(xb, yb)
    loss = loss / acc_steps
    loss.backward() # Backpropagation
```

5 Distributed Data Parallel

Distributed Data Parallel

Distributed Data Parallel (DDP) 目标是进行分布式训练，利用多张显卡并行处理数据，同时保证每张显卡上的模型参数保持同步. 其工作流程为：模型复制 (复制模型到每一张 GPU 上) → 数据切分 (每一轮迭代 GPU 平分数据) → 独立计算 (每张 GPU 独立进行前、反向传播计算梯度) → 梯度同步 (每张 GPU 获得相同的平均梯度) → 统一更新 (每张 GPU 使用平均梯度更新模型).

在部署时，① 使用 $\texttt{int(os.environ.get('RANK', -1))}$ 判断是否处于分布式模式：若能读取到 $\texttt{RANK}$ 则 $\texttt{ddp} = 1$. ② 如果处于分布式模式，则首先使用 $\texttt{init_process_group(backend='nccl')}$

初始化进程组(nccl=NVIDIA Collective Communications Library)，③ 进一步再分配身份变量:

1. NODE: 一个 NODE 对应一台物理机器/服务器
2. RANK: 整个训练集群中进程的唯一编号 (0, 1, 2, ...). Rank 0 通常是主进程
3. LOCAL_RANK: 在该 NODE 中的显卡编号
4. WORLD_SIZE: 总进程数，即总共动用了多少张显卡

④ 除此之外，还需要指定主进程 master_process = ddp_rank == 0，主进程单独负责一些无需所有进程都需要进行的任务，例如打印日志、保存模型、验证等.

```
ddp = int(os.environ.get('RANK', -1)) != -1
if ddp:
    assert torch.cuda.is_available(), "Distributed training requires CUDA"
    init_process_group(backend='nccl') # Initialize the process group for ddp
    ddp_rank = int(os.environ['RANK'])
    ddp_local_rank = int(os.environ['LOCAL_RANK'])
    ddp_world_size = int(os.environ['WORLD_SIZE'])
    device = f'cuda:{ddp_local_rank}'
    torch.cuda.set_device(device)
    master_process = ddp_rank == 0
    print(f"[INFO] DDP initialized. Rank: {ddp_rank}/{ddp_world_size} | Local
        Rank: {ddp_local_rank} | Device: {device}")
else:
    ddp_rank = 0
    ddp_local_rank = 0
    ddp_world_size = 1
    master_process = True
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    print(f"[INFO] Single process training on device: {device}")
```

由于我们使用了多张 GPU，因此 Gradient Accumulation 处的 acc_steps 也需要相应修改:

```
assert total_batch_size % (batch_size * block_size * ddp_world_size) == 0, "Total
    batch size must be divisible by (batch_size * block_size * ddp_world_size)"
acc_steps = total_batch_size // (batch_size * block_size * ddp_world_size)
```

为了将多个孤立运行在不同 GPU 上的模型重新融合为一个整体 (梯度平均)，还需要进行封装:

```
if ddp:
    model = torch.nn.parallel.DistributedDataParallel(model, device_ids=[
        ddp_local_rank])
```

由于我们进 gradient accumulation，因此在这些 micro_step 中我们是不需要不同的 GPU 间相互通信的，只需要完成一个 iteration 再通信:

```
if ddp:
    model.require_backward_grad_sync = (micro_step == acc_steps - 1)
```

同理我们在每个 GPU 上计算的 loss 也需要平均:

```
import torch.distributed as dist
if ddp:
    dist.all_reduce(train_loss, op=dist.ReduceOp.AVG)
```

注意此时由于我们将 model 封装进 DDP，因此我们的优化器需要重新针对我们原本的 model:

```
raw_model = model.module if ddp else model
optimizer = raw_model.configure_optimizers(weight_decay=0.1, learning_rate=lr_max
    , device=device)
```

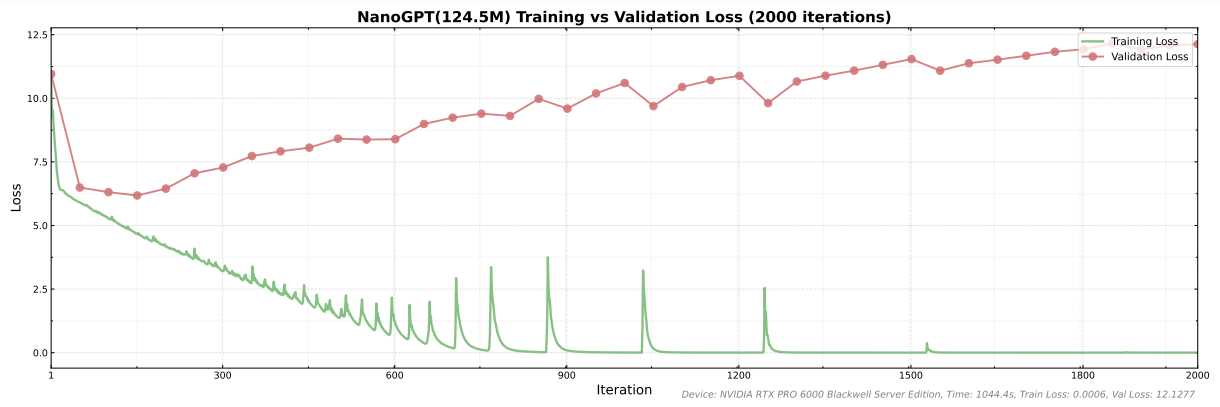
调用多进程分布式训练必须使用专门的启动命令行工具，其中 `standalone` 适用于单机多卡：

```
torchrun --standalone --nproc_per_node=x xxx.py (your file name)
```

6 Final Training Loss – Overfitting

Final Training Loss – Overfitting

在最后的训练中我们可以看到严重的过拟合问题，这是由于我们的数据集太小.



Output

==========
PROMPT: To be, or not to be, that is a question:
==========

[SAMPLE 1]:
To be, or not to be, that is a question: and
Because I would believe to my deeds.

PRINCE EDWARD:
Is it, uncle?
Where is the bloody deed?
Might have I spent.

LEONTESTER:
Because my prayers here
———————————

[SAMPLE 2]:
To be, or not to be, that is a question: but,
Very well that's love as he was commanded
Or in the nose is such a life,
I may use in the world goes not
Appear in his removedness from time, but I now.

Crown fear
———————————

7 Training Dataset

Training Dataset

由于 GPT-2 和GPT-3 中的数据集都未公开，因此我们选用一些公开的高质量数据集，例如：① RedPajama [4] 或其去重版本 SlimPajama ② FineWeb 或其子数据集 FineWeb-Edu. 在本次试验中，我们将选用 `sample – 10BT` 数据集，其包含了 10B gpt2 tokens. 在下载数据时，由于我们的数据集约有 20G，因此若将其整体读取为一个文件，那么很可能造成显存溢出，所以我们将其分片读取，每一片 (即每一个文件) 中有 100M tokens:

```
...
shard_size = int(1e8)
...
```

最终我们的文件数为 $\frac{10B}{100M} = 100$ 个：

```
ls data/FineWeb-edu-10BT/ | wc -l
```

Output

100

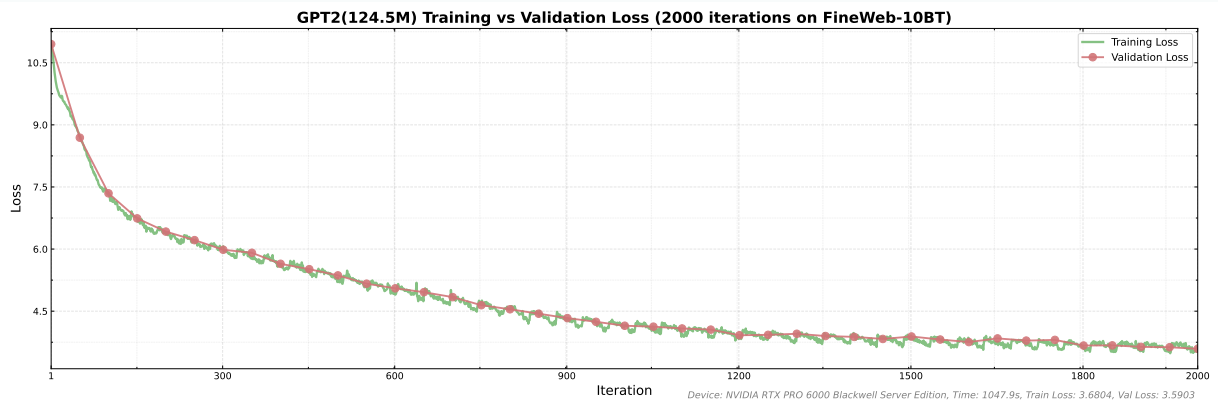
8 Training

Training

由于我们每一个 step 要处理的 `total_batch_size` $= 2^{19} = 524288$，因此我们处理完所有的 token 大约需要 $\frac{10B}{2^{19}} \approx 19073$ steps，而根据 GPT-3 所述，warm up 针对前 375M token，也就是 $\frac{375M}{2^{19}} \approx 715$ steps.

```
lr_max = 6e-4
lr_min = lr_max * 0.1
warm_steps = 715
max_steps = 19073
```

可以看到由于数据集很大，这次训练 2000 step 没有出现过拟合的现象：



9 HellaSwag for Evaluation

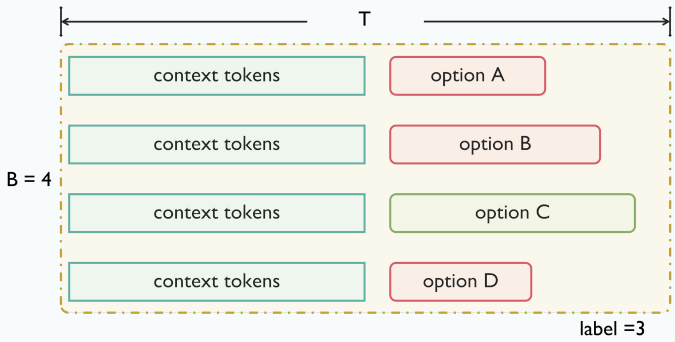
HellaSwag for Evaluation

HellaSwag[5] 是一个著名的常识推理 (Commonsense Reasoning) 评测数据集，目前最高准确率已经可以超过 95%. 每一个题目包含背景描述 (content) 和 4 个选项，例如：

HellaSwag Example

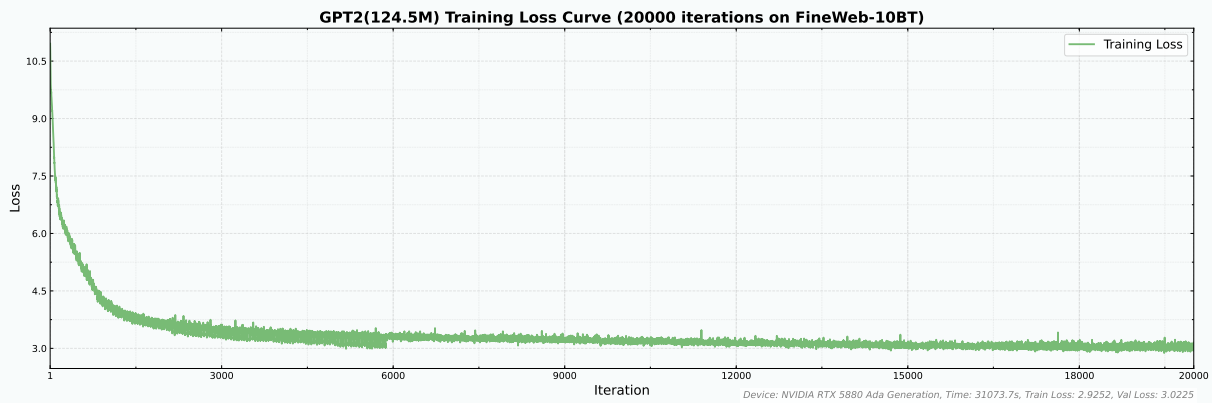
A woman is outside with a bucket and a dog. The dog is running around trying to avoid a bath. She...
A. rinses the bucket off with soap and blow dry the dog's head.
B. uses a hose to keep it from getting soapy.
C. gets the dog wet, then it runs away again. [✓]
D. gets into a bath tub with the dog.

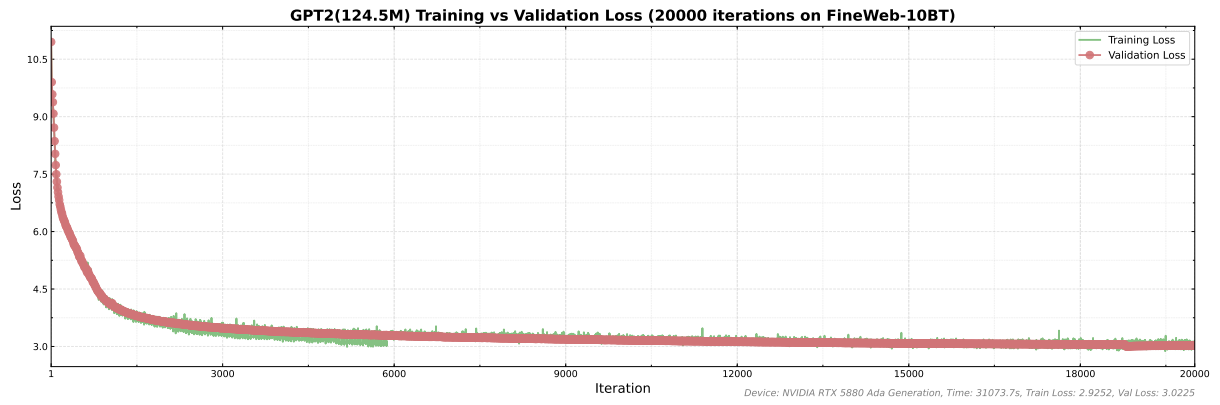
由于 GPT-2 并不能直接做选择题，因此我们实际上是把选项变成"续写"：① 将 B (Batch Size) 设为 4，每一条数据是 Context Tokens + Option ② 遍历这 4 条文本，计算选项部分每一个 Token 的平均对数概率，然后选取概率最高着作为答案.



10 Final Results

Final Results





Output

==========
HELLASWAG RESULTS
Model: GPT2_124.5M_20000iterations.pth
Total Samples: 10042
Accuracy (norm): 0.3026
==========

Notes

在 Andrej Karpathy 的教程中可以看到，如果增加 epoch 数，即重复地使用完整的数据集进行训练会提高最终的 HellaSwag accuracy. 并且如果增大 learning rate 也可以提高训练速度.

Last updated: April 13, 2026

References

[1] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[3] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[4] Maurice Weber, Daniel Y Fu, Quentin Anthony, Yonatan Oren, Shane Adams, Anton Alexandrov, Xiaozhong Lyu, Huu Nguyen, Xiaozhe Yao, Virginia Adams, et al. Redpajama: an open dataset for training large language models. *Advances in neural information processing systems*, 37:116462–116492, 2024.

[5] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 4791–4800, 2019.